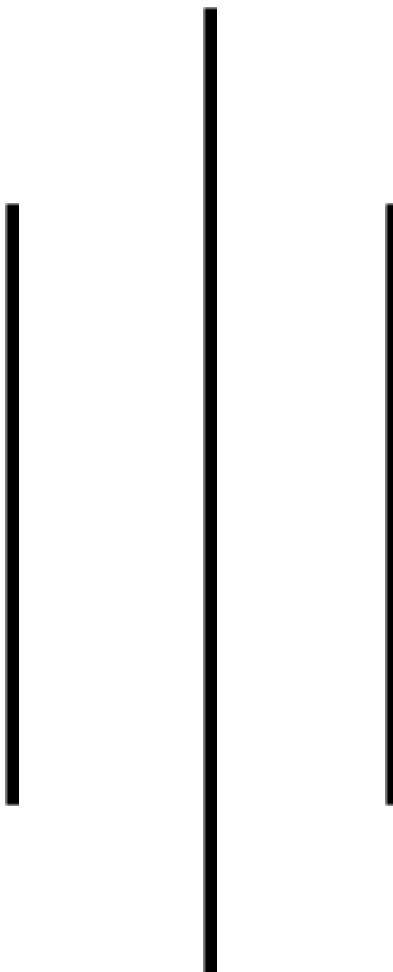


PATAN NISTHA CAMPUS

Gwarko, Lalitpur
[Affiliated to TU]



TU Registration Number:6-2-505-26-2020

Submitted By: Name: Roshan Dhungana Year/Semester: VIII Program: BCA Symbol. No.: 50502025	Submitted To: Ms. Sabita Rajbanshi Lecturer/Faculty Member
---	---

Table of Content

1. Load any dataset and apply K-Means clustering. Compare the cluster labels with the actual class labels using a confusion matrix. Plot the cluster centroids and explain their significance in the context of the dataset.	2
2. Apply Agglomerative Clustering using Euclidean distance on a suitable dataset. Assign cluster labels and visualize the results using a scatter plot. Compare the predicted clusters with the actual class labels. Analyze how varying the number of clusters influences the clustering outcome and the quality of the results.	5
3. Perform Principal Component Analysis (PCA) on any dataset and reduce its dimensionality to two principal components. Visualize the resulting 2D data. Then, train a classification algorithm on both the original dataset and the PCA-transformed dataset, and compare their performance to evaluate the impact of dimensionality reduction.	8
5. Using an imbalanced dataset evaluate the model's performance by calculating precision, recall, and F1-score, and discuss how class imbalance dataset impacts these metrics.	15
6. Load a dataset (any open-source dataset) and perform basic statistical analysis and visualization. Identify the types of data and draw the learning curve.	18
9. Implement linear regression using a dataset like housing prices. Plot the best-fit line and compute Mean Squared Error (MSE).	29
10. Train a decision tree classifier on a dataset. Visualize the tree and explain how it makes decisions.	32
11. Use SVM to classify a dataset with clear margins. Visualize the margin and support vectors	35
13. Implement KNN on a dataset. Experiment with different values of K and analyze how the model's accuracy and decision boundaries change.	41

1. Load any dataset and apply K-Means clustering. Compare the cluster labels with the actual class labels using a confusion matrix. Plot the cluster centroids and explain their significance in the context of the dataset.

```
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn import datasets
from sklearn.cluster import KMeans
from sklearn.metrics import confusion_matrix, accuracy_score
from sklearn.decomposition import PCA

----
# Step 1: Load Dataset
----
iris = datasets.load_iris()
X = iris.data
y_actual = iris.target
class_names = iris.target_names
num_classes = len(class_names)

print("Iris dataset loaded successfully.")

----
# Step 2: Apply K-Means Clustering
----
kmeans_model = KMeans(
    n_clusters=num_classes,
    init="k-means++",
    n_init=10,
    random_state=42
)

cluster_labels = kmeans_model.fit_predict(X)
centroids = kmeans_model.cluster_centers_

----
# Step 3: Function to map cluster IDs -> true labels
----
def remap_cluster_labels(true_labels, predicted_labels, k):
    """Maps cluster labels to their closest actual class using majority voting."""

    label_map = {}
    for cid in range(k):
        cluster_mask = (predicted_labels == cid)
        most_common = np.bincount(true_labels[cluster_mask]).argmax()
        label_map[cid] = most_common
```

```

    return np.array([label_map[label] for label in predicted_labels])

mapped_labels = remap_cluster_labels(y_actual, cluster_labels, num_classes)

----
# Step 4: Confusion Matrix
----
cm = confusion_matrix(y_actual, mapped_labels)

plt.figure(figsize=(8, 6))
sns.heatmap(cm, annot=True, fmt="d", cmap="Blues",
            xticklabels=class_names, yticklabels=class_names)
plt.title("Confusion Matrix: K-Means vs Actual Labels")
plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.show()

print(f"Accuracy after label mapping: {accuracy_score(y_actual, mapped_labels):.4f}")

----
# Step 5: Visualizing Clusters & Centroids with PCA
----
pca = PCA(n_components=2)
X_reduced = pca.fit_transform(X)
centroids_reduced = pca.transform(centroids)

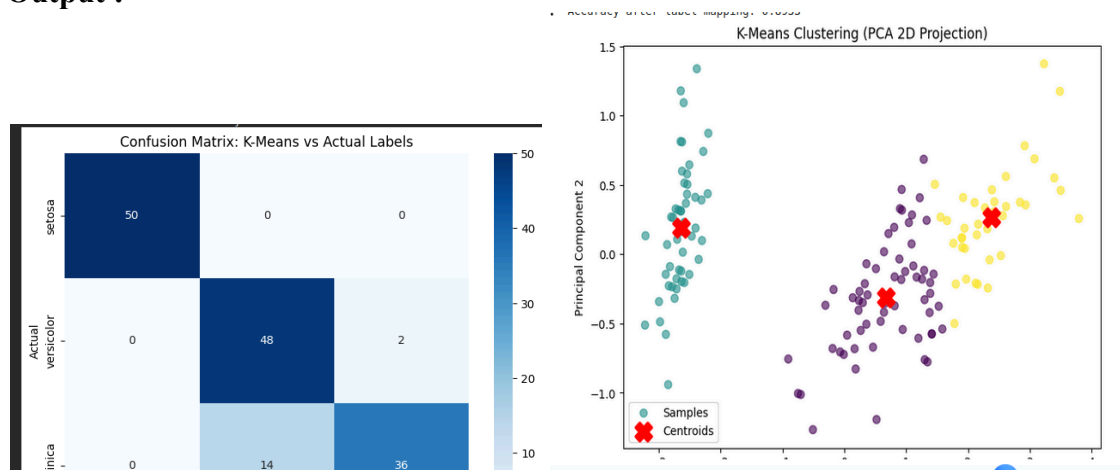
plt.figure(figsize=(8, 6))
plt.scatter(
    X_reduced[:, 0],
    X_reduced[:, 1],
    c=cluster_labels,
    cmap="viridis",
    s=45,
    alpha=0.6,
    label="Samples"
)

plt.scatter(
    centroids_reduced[:, 0],
    centroids_reduced[:, 1],
    marker="X",
    s=220,
    linewidths=2.5,
    color="red",
    label="Centroids"
)

```

```
plt.title("K-Means Clustering (PCA 2D Projection)")
plt.xlabel("Principal Component 1")
plt.ylabel("Principal Component 2")
plt.legend()
plt.show()
```

Output :



2. Apply Agglomerative Clustering using Euclidean distance on a suitable dataset. Assign cluster labels and visualize the results using a scatter plot. Compare the predicted clusters with the actual class labels. Analyze how varying the number of clusters influences the clustering outcome and the quality of the results.

```
import matplotlib.pyplot as plt
import pandas as pd
import numpy as np

from sklearn.datasets import load_iris
from sklearn.cluster import AgglomerativeClustering
from sklearn.decomposition import PCA
from sklearn.metrics import adjusted_rand_score

# Step 1: Load Dataset

iris = load_iris()
X = iris.data
y_true = iris.target

print("Iris dataset loaded successfully.")

# Step 2: Reduce to 2D for plotting using PCA

pca = PCA(n_components=2)
X_proj = pca.fit_transform(X)

# For storing evaluation scores
evaluation_scores = []

# We will test the clustering for these k values
cluster_options = [2, 3, 4, 5]

# Step 3: Perform Agglomerative Clustering for different k

plt.figure(figsize=(12, 10))
plot_position = 1

for k in cluster_options:

    # Using 'ward' linkage which relies on Euclidean distance
```

```

aggl_model = AgglomerativeClustering(
    n_clusters=k,
    linkage="ward"
)

predicted_labels = aggl_model.fit_predict(X)

# Compute how well clusters match the true classes
ari_score = adjusted_rand_score(y_true, predicted_labels)
evaluation_scores.append((k, ari_score))

##
# Visualization
##
plt.subplot(2, 2, plot_position)
plt.scatter(
    X_proj[:, 0],
    X_proj[:, 1],
    c=predicted_labels,
    cmap="viridis",
    s=45
)

plt.title(f"k = {k} | ARI = {ari_score:.3f}")
plt.xlabel("PC 1")
plt.ylabel("PC 2")

plot_position += 1

plt.tight_layout()
plt.show()

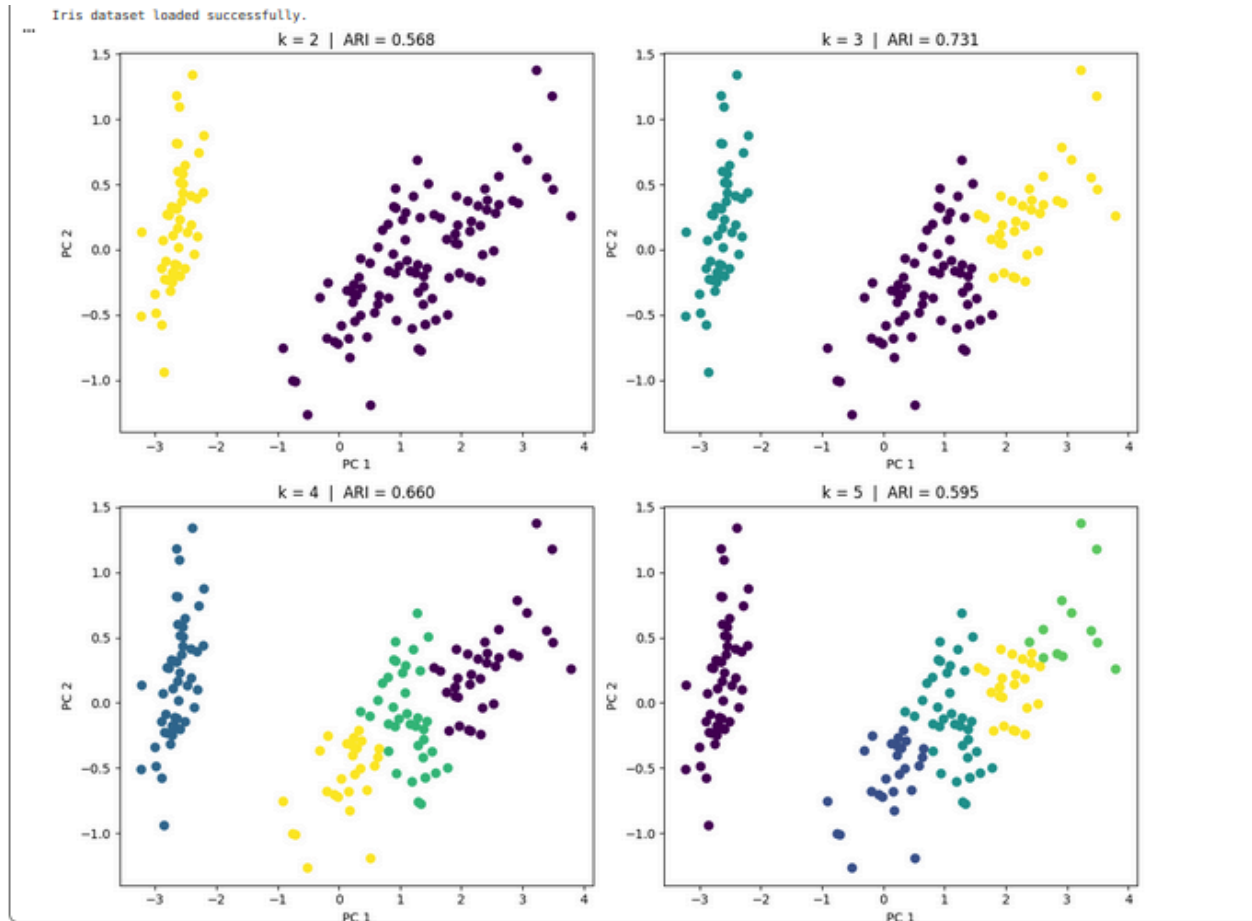
# Step 4: Display ARI scores for all k values

result_df = pd.DataFrame(
    evaluation_scores,
    columns=["Number of Clusters", "Adjusted Rand Index (ARI)"]
)

print("\nComparison of clustering quality for different k values:")
print(result_df)

```

Output :



Comparison of clustering quality for different k values:

	Number of Clusters	Adjusted Rand Index (ARI)
0	2	0.568116
1	3	0.731199
2	4	0.660217
3	5	0.595023

3. Perform Principal Component Analysis (PCA) on any dataset and reduce its dimensionality to two principal components. Visualize the resulting 2D data. Then, train a classification algorithm on both the original dataset and the PCA-transformed dataset, and compare their performance to evaluate the impact of dimensionality reduction.

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
```

```
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.decomposition import PCA
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score
```

```
# Step 1: Load the dataset
```

```
iris = load_iris()
X = iris.data
y = iris.target
```

```
print("Iris dataset loaded successfully.")
print(f"Number of samples: {X.shape[0]}, Number of features: {X.shape[1]}")
```

```
# Step 2: Split data into train and test sets
```

```
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.3, random_state=42
)
```

```
# Standardize features for better model performance
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)
```

Step 3: Apply PCA for dimensionality reduction to 2 components

```
pca = PCA(n_components=2)
X_train_pca = pca.fit_transform(X_train_scaled)
X_test_pca = pca.transform(X_test_scaled)

# 2D visualization of PCA-transformed data
plt.figure(figsize=(6, 5))
plt.scatter(X_train_pca[:, 0], X_train_pca[:, 1], c=y_train, cmap="viridis", s=50, alpha=0.7)
plt.xlabel("Principal Component 1")
plt.ylabel("Principal Component 2")
plt.title("2D Visualization after PCA")
plt.grid(True)
plt.show()
```

Step 4: Train Logistic Regression on original dataset

```
clf_original = LogisticRegression(max_iter=500, random_state=42)
clf_original.fit(X_train_scaled, y_train)
y_pred_original = clf_original.predict(X_test_scaled)
acc_original = accuracy_score(y_test, y_pred_original)
```

Step 5: Train Logistic Regression on PCA-transformed dataset

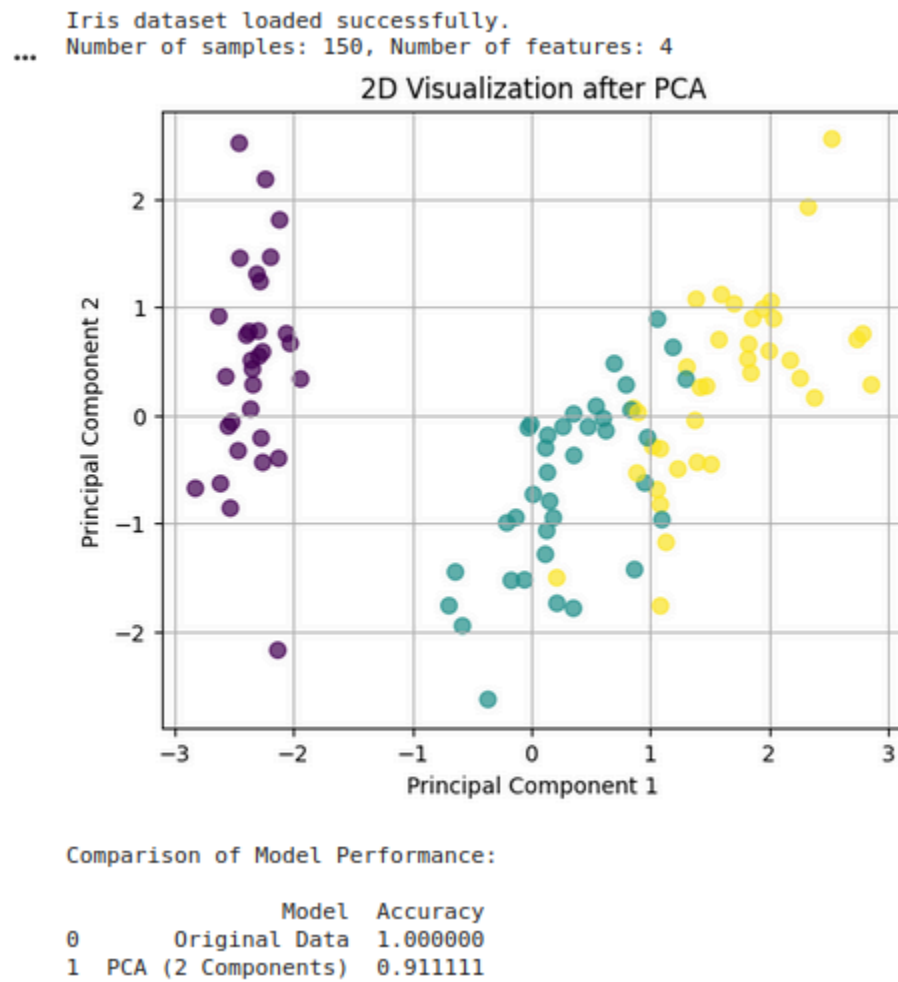
```
clf_pca = LogisticRegression(max_iter=500, random_state=42)
clf_pca.fit(X_train_pca, y_train)
y_pred_pca = clf_pca.predict(X_test_pca)
acc_pca = accuracy_score(y_test, y_pred_pca)
```

Step 6: Compare performance

```
performance_df = pd.DataFrame({
    "Model": ["Original Data", "PCA (2 Components)"],
    "Accuracy": [acc_original, acc_pca]
})

print("\nComparison of Model Performance:\n")
print(performance_df)
```

Output :



4. Using a binary classification task on any balanced dataset, train a classifier and create its confusion matrix. Compute sensitivity (recall) and specificity from the confusion matrix, and plot the ROC curve along with the AUC score

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import confusion_matrix, roc_curve, auc

# Step 1: Load dataset

data = load_breast_cancer()
X = data.data
y = data.target # 0 = malignant, 1 = benign

print("Breast Cancer dataset loaded successfully.")
print(f"Number of samples: {X.shape[0]}, Number of features: {X.shape[1]}")

# Step 2: Split into training and testing sets

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.3, random_state=42, stratify=y
)

# Standardize features
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

# Step 3: Train Logistic Regression Classifier
```

```

clf = LogisticRegression(max_iter=2000, random_state=42)
clf.fit(X_train_scaled, y_train)

# Predictions
y_pred = clf.predict(X_test_scaled)
y_prob = clf.predict_proba(X_test_scaled)[:, 1] # Probability for
positive class

# Step 4: Confusion Matrix

cm = confusion_matrix(y_test, y_pred)
TN, FP, FN, TP = cm.ravel()

# Compute Sensitivity (Recall) and Specificity
sensitivity = TP / (TP + FN)
specificity = TN / (TN + FP)

print("\nConfusion Matrix:\n", cm)
print(f"Sensitivity (Recall): {sensitivity:.3f}")
print(f"Specificity: {specificity:.3f}")

# Step 5: ROC Curve & AUC

fpr, tpr, thresholds = roc_curve(y_test, y_prob)
roc_auc = auc(fpr, tpr)

print(f"AUC Score: {roc_auc:.3f}")

plt.figure(figsize=(6, 5))
plt.plot(fpr, tpr, color="blue", lw=2, label=f"AUC = {roc_auc:.3f}")
plt.plot([0, 1], [0, 1], linestyle="--", color="gray")
plt.xlabel("False Positive Rate")
plt.ylabel("True Positive Rate")
plt.title("ROC Curve for Logistic Regression")
plt.legend(loc="lower right")
plt.grid(True)
plt.show()

```


Output :

```
Breast Cancer dataset loaded successfully.  
... Number of samples: 569, Number of features: 30
```

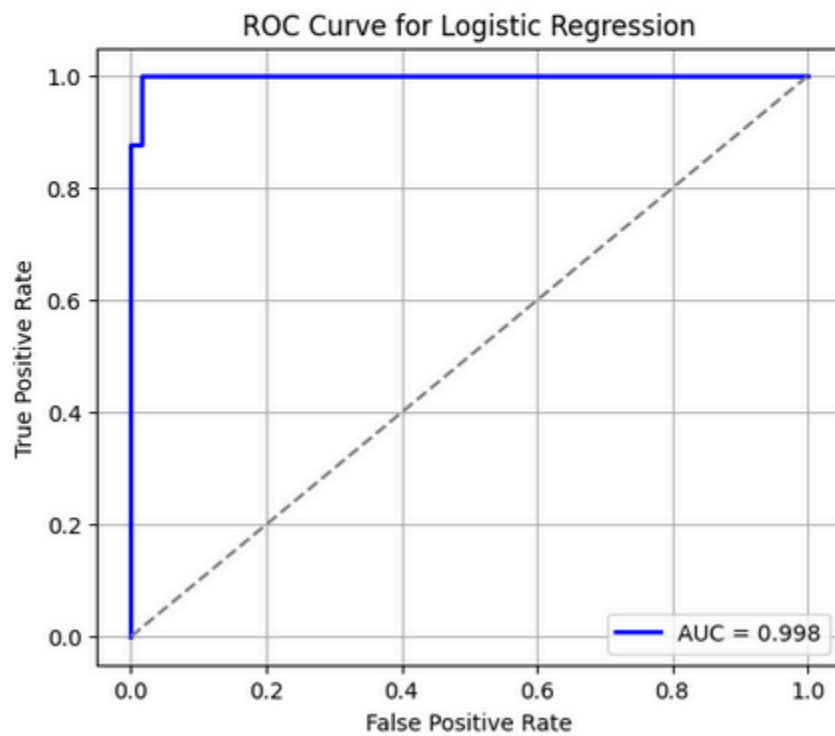
Confusion Matrix:

```
[[ 63   1]  
 [  1 106]]
```

Sensitivity (Recall): 0.991

Specificity: 0.984

AUC Score: 0.998



5. Using an imbalanced dataset evaluate the model's performance by calculating precision, recall, and F1-score, and discuss how class imbalance dataset impacts these metrics.

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

from sklearn.datasets import make_classification
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import (
    confusion_matrix,
    precision_score,
    recall_score,
    f1_score,
    classification_report
)

# Step 1: Generate imbalanced dataset

X, y = make_classification(
    n_samples=2000,
    n_features=10,
    n_informative=5,
    n_redundant=2,
    n_classes=2,
    weights=[0.95, 0.05], # 95% class 0, 5% class 1
    random_state=42
)

print("Imbalanced dataset generated.")
print(f"Class distribution: {np.bincount(y)}") # 0 = majority, 1 = minority

# Step 2: Split dataset into train and test sets

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.3, random_state=42, stratify=y
)

# Step 3: Train Logistic Regression classifier

clf = LogisticRegression(max_iter=2000, random_state=42)
```

```

clf.fit(X_train, y_train)

# Predictions
y_pred = clf.predict(X_test)

# Step 4: Evaluate performance

cm = confusion_matrix(y_test, y_pred)
precision = precision_score(y_test, y_pred)
recall = recall_score(y_test, y_pred)
f1 = f1_score(y_test, y_pred)

print("\nConfusion Matrix:\n", cm)
print(f"\nPrecision: {precision:.3f}")
print(f"Recall: {recall:.3f}")
print(f"F1-Score: {f1:.3f}")

print("\nClassification Report:\n", classification_report(y_test, y_pred))

# Step 5: Interpretation

print("Note:")
print("- Precision measures the proportion of correctly predicted positive cases out of all predicted positives.")
print("- Recall (Sensitivity) measures the proportion of actual positives correctly identified.")
print("- F1-score balances precision and recall.")
print("- On imbalanced datasets, accuracy alone is misleading; metrics like precision, recall, and F1-score are more informative.")

```

Output :

```
... Imbalanced dataset generated.
    Class distribution: [1892 108]

Confusion Matrix:
[[566  2]
 [ 31  1]]

Precision: 0.333
Recall: 0.031
F1-Score: 0.057

Classification Report:
              precision    recall  f1-score   support

     0       0.95         1.00         0.97         568
     1       0.33         0.03         0.06          32

   accuracy                   0.94         600
  macro avg       0.64         0.51         0.51         600
 weighted avg       0.92         0.94         0.92         600

Note:
- Precision measures the proportion of correctly predicted positive cases out of all predicted positives.
- Recall (Sensitivity) measures the proportion of actual positives correctly identified.
- F1-score balances precision and recall.
- On imbalanced datasets, accuracy alone is misleading; metrics like precision, recall, and F1-score are more informative.
```

6. Load a dataset (any open-source dataset) and perform basic statistical analysis and visualization. Identify the types of data and draw the learning curve.

```
import pandas as pd
import numpy as np
import seaborn as sns
import matplotlib.pyplot as plt

from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split, learning_curve
from sklearn.ensemble import RandomForestClassifier

# Step 1: Load dataset and create DataFrame

iris = load_iris()
df = pd.DataFrame(data=iris.data, columns=iris.feature_names)
df['target'] = iris.target

print("First 5 rows of the dataset:")
print(df.head())

# Step 2: Identify data types

print("\nData Types:")
print(df.dtypes)

# Step 3: Basic statistical analysis

print("\nStatistical Summary:")
print(df.describe())

print("\nTarget Value Counts:")
print(df['target'].value_counts())

# Step 4: Visualizations

# Pairplot to visualize feature relationships
sns.pairplot(df, hue='target')
plt.show()

# Correlation heatmap
plt.figure(figsize=(8,6))
```

```
sns.heatmap(df.corr(), annot=True, cmap='coolwarm')
plt.title('Feature Correlation Heatmap')
plt.show()
```

```
# Boxplot for each feature
plt.figure(figsize=(10,6))
sns.boxplot(data=df)
plt.title('Boxplot of Features')
plt.show()
```

Step 5: Prepare features and labels

```
X = df[iris.feature_names]
y = df['target']
```

```
# Split dataset
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)
```

Step 6: Initialize classifier and generate learning curve

```
clf = RandomForestClassifier(random_state=42)

train_sizes, train_scores, test_scores = learning_curve(
    clf, X, y, cv=5, n_jobs=-1,
    train_sizes=np.linspace(0.1, 1.0, 10),
    scoring='accuracy'
)
```

```
# Compute mean and standard deviation
train_mean = np.mean(train_scores, axis=1)
train_std = np.std(train_scores, axis=1)
test_mean = np.mean(test_scores, axis=1)
test_std = np.std(test_scores, axis=1)
```

Step 7: Plot learning curve

```
plt.figure(figsize=(10,6))
plt.plot(train_sizes, train_mean, 'o-', color='blue', label='Training Score')
plt.plot(train_sizes, test_mean, 'o-', color='green', label='Cross-Validation Score')
```

Fill standard deviation area

```
plt.fill_between(train_sizes, train_mean - train_std, train_mean + train_std, alpha=0.1,
color='blue')
plt.fill_between(train_sizes, test_mean - test_std, test_mean + test_std, alpha=0.1,
color='green')

plt.title('Learning Curve for Random Forest Classifier')
plt.xlabel('Training Set Size')
plt.ylabel('Accuracy')
plt.legend(loc='best')
plt.grid(True)
plt.show()
```

Output :

First 5 rows of the dataset:

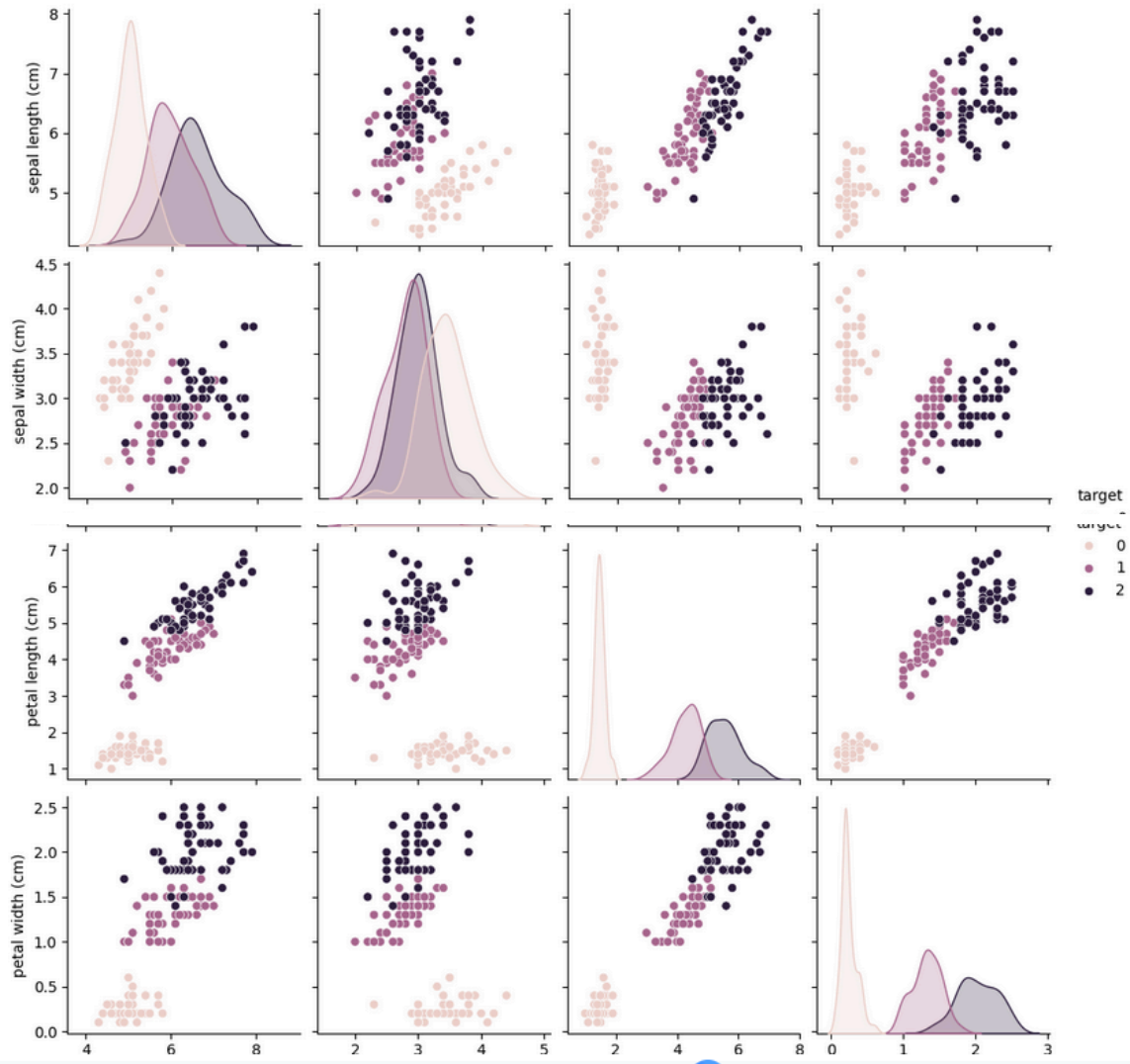
```
plt.grid(True)
plt.show()
```

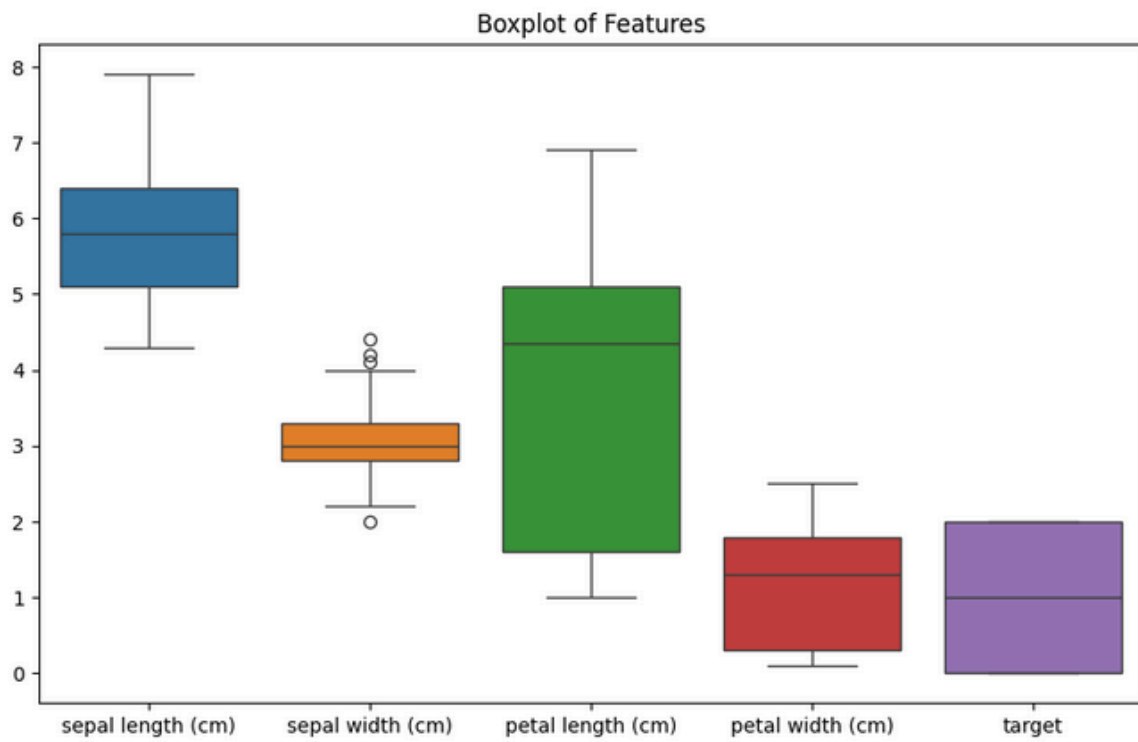
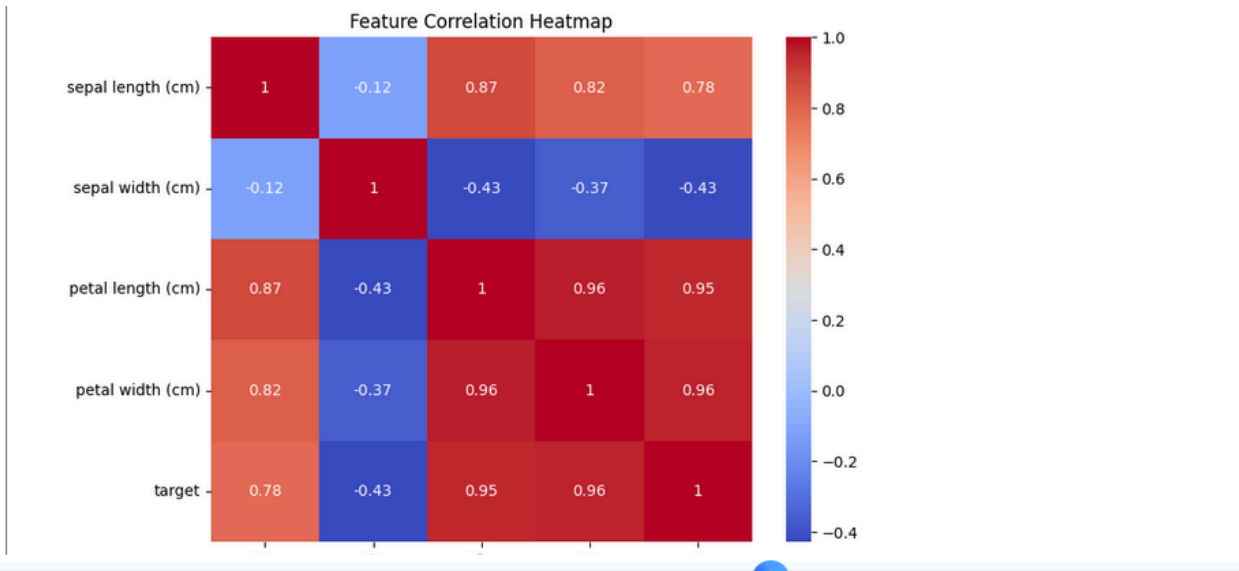
```
*** First 5 rows of the dataset:
      sepal length (cm)  sepal width (cm)  petal length (cm)  petal width (cm)  \
0                5.1             3.5             1.4             0.2
1                4.9             3.0             1.4             0.2
2                4.7             3.2             1.3             0.2
3                4.6             3.1             1.5             0.2
4                5.0             3.6             1.4             0.2

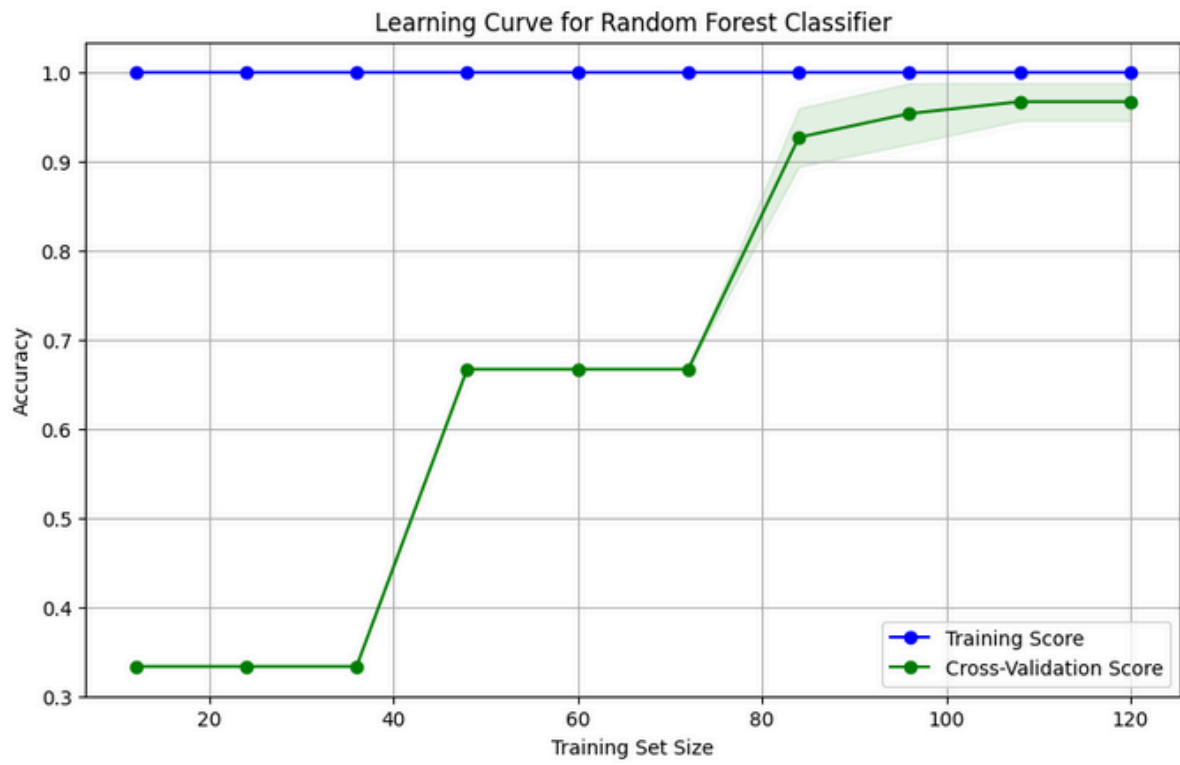
      target
0          0
1          0
2          0
3          0
4          0

Data Types:
sepal length (cm)    float64
sepal width (cm)     float64
petal length (cm)    float64
petal width (cm)     float64
target               int64
dtype: object
```

```
4 50
2 50
Name: count, dtype: int64
```







7. Simulate and visualize the effect of underfitting and overfitting using polynomial regression on a dataset. Discuss the impact on bias and variance.

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.preprocessing import PolynomialFeatures
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error

# Step 1: Generate synthetic dataset

np.random.seed(42)
X = np.linspace(0, 10, 100).reshape(-1, 1) # 100 points, reshaped for sklearn
y = np.sin(X).ravel() + np.random.normal(0, 0.3, X.shape[0]) # sine function + noise

# Step 2: Define polynomial degrees

degrees = [1, 3, 15] # low = underfit, medium = good fit, high = overfit

# Plot the dataset
plt.figure(figsize=(12,6))
plt.scatter(X, y, color='black', label='Data')

# Step 3: Fit polynomial regression models and plot predictions

for degree in degrees:
    # Create polynomial features
    poly = PolynomialFeatures(degree=degree)
```

```

X_poly = poly.fit_transform(X)

# Fit linear regression on polynomial features
model = LinearRegression()
model.fit(X_poly, y)

# Predict
y_pred = model.predict(X_poly)

# Compute mean squared error
mse = mean_squared_error(y, y_pred)

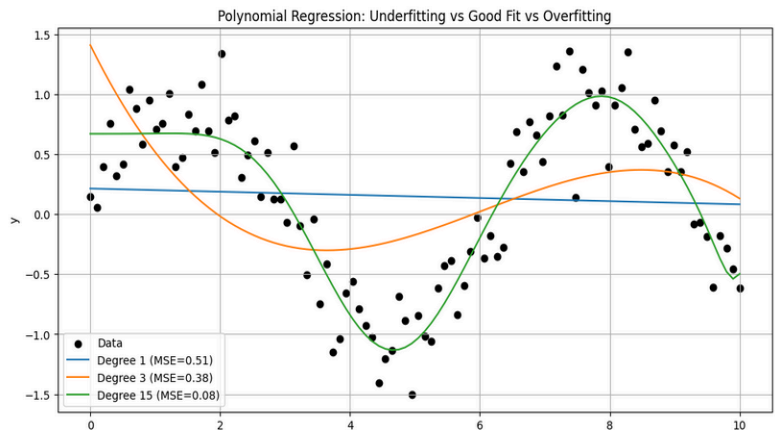
# Plot predictions
plt.plot(X, y_pred, label=f'Degree {degree} (MSE={mse:.2f})')

# Step 4: Final plot settings

plt.title('Polynomial Regression: Underfitting vs Good Fit vs Overfitting')
plt.xlabel('X')
plt.ylabel('y')
plt.legend()
plt.grid(True)
plt.show()

```

Output:



8. Introduce random noise into a dataset and observe its effect on model performance. Use a simple classifier or regression model to demonstrate.

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error

# Step 1: Generate clean dataset

np.random.seed(42)
X = np.linspace(0, 10, 100).reshape(-1, 1) # Features
y_clean = 2 * X.flatten() + 3 # Linear relationship:  $y = 2x + 3$ 

# Step 2: Define noise levels

noise_levels = [0, 2, 5] # No noise, moderate noise, high noise

# Step 3: Visualize the effect of noise

plt.figure(figsize=(12,6))

for noise in noise_levels:
    # Add Gaussian noise
    y_noisy = y_clean + np.random.normal(scale=noise, size=y_clean.shape)

    # Fit Linear Regression on noisy data
    model = LinearRegression()
    model.fit(X, y_noisy)
    y_pred = model.predict(X)

    # Compute mean squared error
    mse = mean_squared_error(y_noisy, y_pred)

    # Plot noisy data points
    plt.scatter(X, y_noisy, alpha=0.5, label=f'Noise={noise}')
```

```
# Plot model prediction  
plt.plot(X, y_pred, label=f'Prediction (Noise={noise}, MSE={mse:.2f})')
```

Step 4: Final plot settings

```
plt.title('Effect of Random Noise on Linear Regression Model')  
plt.xlabel('X')  
plt.ylabel('y')  
plt.legend()  
plt.grid(True)  
plt.show()
```

Output :



9. Implement linear regression using a dataset like housing prices. Plot the best-fit line and compute Mean Squared Error (MSE).

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.datasets import fetch_california_housing
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error

# Step 1: Load California Housing dataset

data = fetch_california_housing()
X = data.data[:, 0].reshape(-1, 1) # Feature: MedInc (median income)
y = data.target # Target: House prices

print(f"Number of samples: {X.shape[0]}, Feature used: MedInc")

# Step 2: Split data into training and testing sets

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

# Step 3: Train Linear Regression model

model = LinearRegression()
model.fit(X_train, y_train)
```

```
# Predictions
```

```
y_pred = model.predict(X_test)
```

```
# Compute Mean Squared Error
```

```
mse = mean_squared_error(y_test, y_pred)
```

```
print(f"Mean Squared Error (MSE): {mse:.4f}")
```

```
# Step 4: Plot best-fit line and actual data
```

```
plt.figure(figsize=(10,6))
```

```
plt.scatter(X_test, y_test, color='blue', alpha=0.5, label='Actual Prices')
```

```
plt.plot(X_test, y_pred, color='red', label='Best-Fit Line')
```

```
plt.title('Linear Regression: House Prices vs Median Income')
```

```
plt.xlabel('Median Income (10k USD)')
```

```
plt.ylabel('House Price (100k USD)')
```

```
plt.legend()
```

```
plt.grid(True)
```

```
plt.show()
```

Output :



10. Train a decision tree classifier on a dataset. Visualize the tree and explain how it makes decisions.

```
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.datasets import load_iris
from sklearn.tree import DecisionTreeClassifier, plot_tree
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score

# Step 1: Load Iris dataset

iris = load_iris()
X = iris.data
y = iris.target
feature_names = iris.feature_names
class_names = iris.target_names

print(f'Dataset samples: {X.shape[0]}, Features: {X.shape[1]}")

# Step 2: Split dataset into training and testing sets

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)
```

Step 3: Train Decision Tree Classifier

```
clf = DecisionTreeClassifier(max_depth=3, random_state=42) # Limit depth for clarity
clf.fit(X_train, y_train)
```

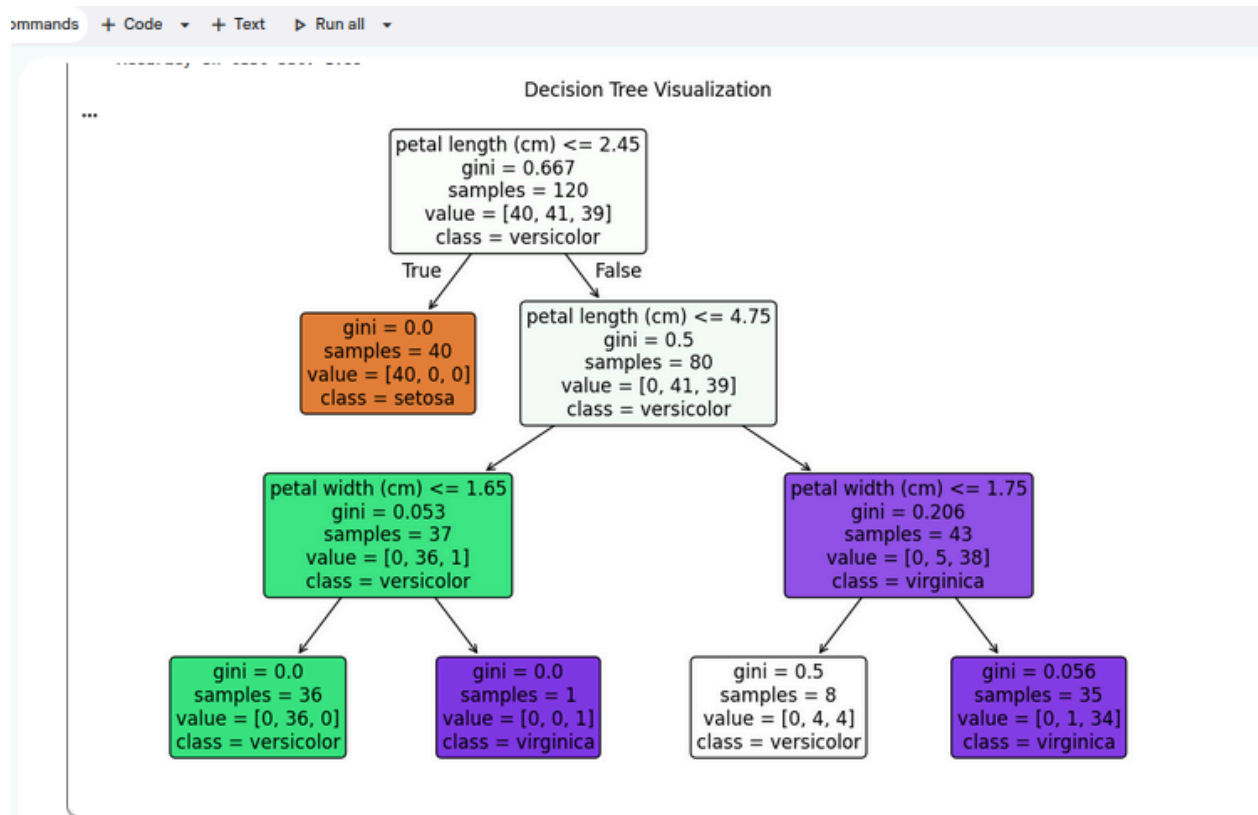
Predict and evaluate

```
y_pred = clf.predict(X_test)
accuracy = accuracy_score(y_test, y_pred)
print(f'Accuracy on test set: {accuracy:.2f}')
```

Step 4: Visualize the decision tree

```
plt.figure(figsize=(12,8))
plot_tree(
    clf,
    feature_names=feature_names,
    class_names=class_names,
    filled=True,
    rounded=True,
    fontsize=12
)
plt.title('Decision Tree Visualization')
plt.show()
```

Output :



11. Use SVM to classify a dataset with clear margins. Visualize the margin and support vectors

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.datasets import make_blobs
from sklearn.svm import SVC

# Step 1: Generate a simple linearly separable dataset

X, y = make_blobs(
    n_samples=50,
    centers=2,
    random_state=42,
    cluster_std=1.0
)

# Step 2: Train SVM classifier

svm_clf = SVC(kernel='linear', C=1000) # High C for hard margin
svm_clf.fit(X, y)

# Step 3: Define function to plot decision boundary and margins

def plot_svc_decision_boundary(model, X, y):
    plt.figure(figsize=(10,6))

    # Plot data points
    plt.scatter(X[:, 0], X[:, 1], c=y, s=50, cmap='bwr', edgecolors='k', label='Data points')

    # Highlight support vectors
    plt.scatter(model.support_vectors_[:, 0], model.support_vectors_[:, 1],
                s=150, facecolors='none', edgecolors='k', label='Support Vectors')

    # Create grid to evaluate model
    xlim = plt.xlim()
    ylim = plt.ylim()
    xx = np.linspace(xlim[0]-1, xlim[1]+1, 200)
    yy = np.linspace(ylim[0]-1, ylim[1]+1, 200)
    YY, XX = np.meshgrid(yy, xx)
    xy = np.vstack([XX.ravel(), YY.ravel()]).T
    Z = model.decision_function(xy).reshape(XX.shape)
```

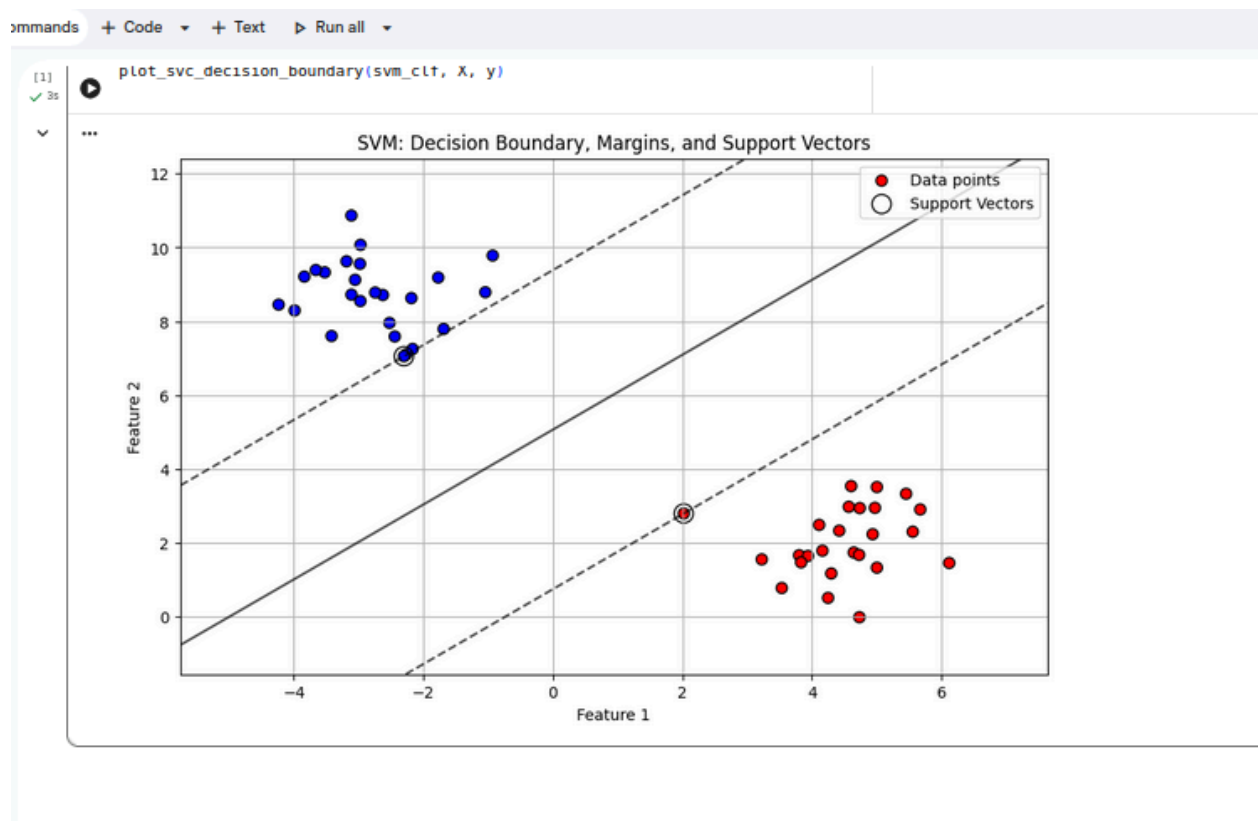
```
# Plot decision boundary and margins
plt.contour(XX, YY, Z, colors='k', levels=[-1, 0, 1], alpha=0.7,
            linestyle=['--', '-', '--'])

# Final plot settings
plt.title('SVM: Decision Boundary, Margins, and Support Vectors')
plt.xlabel('Feature 1')
plt.ylabel('Feature 2')
plt.legend()
plt.grid(True)
plt.show()

# Step 4: Plot decision boundary, margins, and support vectors

plot_svc_decision_boundary(svm_clf, X, y)
```

Output :



12. Train multiple classifiers (Perceptron, Decision Tree, SVM) on the same dataset and compare their performance metrics (accuracy, precision, recall)

```
import pandas as pd
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import Perceptron
from sklearn.tree import DecisionTreeClassifier
from sklearn.svm import SVC
from sklearn.metrics import accuracy_score, precision_score, recall_score

# Step 1: Load Iris dataset

iris = load_iris()
X = iris.data
y = iris.target
print(f'Dataset: {X.shape[0]} samples, {X.shape[1]} features, {len(set(y))} classes")

# Step 2: Split dataset into training and testing sets

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)

# Step 3: Standardize features for Perceptron and SVM

scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
```

```
X_test_scaled = scaler.transform(X_test)
```

```
# Step 4: Define classifiers
```

```
classifiers = {  
    "Perceptron": Perceptron(max_iter=1000, random_state=42),  
    "Decision Tree": DecisionTreeClassifier(max_depth=3, random_state=42),  
    "SVM": SVC(kernel='linear', random_state=42)  
}
```

```
# Step 5: Train, predict, and calculate performance metrics
```

```
results = []
```

```
for name, clf in classifiers.items():  
    # Use scaled data for Perceptron and SVM  
    if name in ["Perceptron", "SVM"]:  
        clf.fit(X_train_scaled, y_train)  
        y_pred = clf.predict(X_test_scaled)  
    else:  
        clf.fit(X_train, y_train)  
        y_pred = clf.predict(X_test)  
  
    accuracy = accuracy_score(y_test, y_pred)  
    precision = precision_score(y_test, y_pred, average='macro')  
    recall = recall_score(y_test, y_pred, average='macro')  
  
    results.append({  
        "Classifier": name,
```

```

    "Accuracy": round(accuracy, 4),
    "Precision": round(precision, 4),
    "Recall": round(recall, 4)
})

```

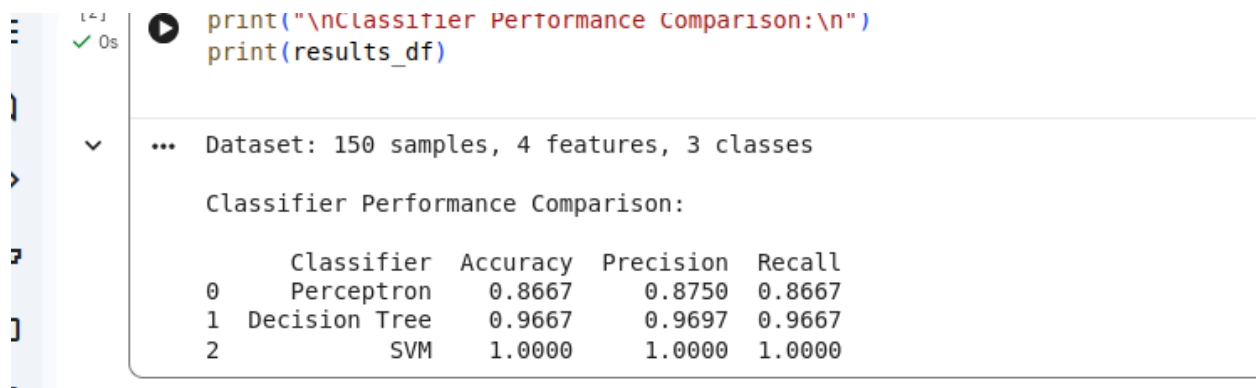
Step 6: Display results

```

results_df = pd.DataFrame(results)
print("\nClassifier Performance Comparison:\n")
print(results_df)

```

Output :



The screenshot shows a Jupyter Notebook interface. On the left, there is a vertical toolbar with icons for running, saving, and other notebook functions. The main area displays the output of a code cell. At the top, there is a status bar indicating '0s' and a green checkmark. Below this, the output is displayed in a monospaced font. It starts with a header line, followed by a line indicating the dataset size, and then a table of classifier performance metrics.

```

print("\nClassifier Performance Comparison:\n")
print(results_df)

```

... Dataset: 150 samples, 4 features, 3 classes

Classifier Performance Comparison:

	Classifier	Accuracy	Precision	Recall
0	Perceptron	0.8667	0.8750	0.8667
1	Decision Tree	0.9667	0.9697	0.9667
2	SVM	1.0000	1.0000	1.0000

13. Implement KNN on a dataset. Experiment with different values of K and analyze how the model's accuracy and decision boundaries change.

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.datasets import make_moons
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import accuracy_score

##
# Step 1: Generate dataset
##
X, y = make_moons(n_samples=300, noise=0.3, random_state=42)

# Split dataset into train and test
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

# Standardize features
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

##
# Step 2: Define function to plot decision boundaries
##
def plot_decision_boundary(clf, X, y, ax, title):
    x_min, x_max = X[:,0].min() - 1, X[:,0].max() + 1
    y_min, y_max = X[:,1].min() - 1, X[:,1].max() + 1
    xx, yy = np.meshgrid(np.linspace(x_min, x_max, 300),
                          np.linspace(y_min, y_max, 300))
    Z = clf.predict(np.c_[xx.ravel(), yy.ravel()]).reshape(xx.shape)

    ax.contourf(xx, yy, Z, alpha=0.3, colors=['lightgray', 'white'])
    # Plot data points with markers for B/W printing
    ax.scatter(X[y==0,0], X[y==0,1], marker='o', facecolors='none',
               edgecolors='black', label='Class 0')
```

```

    ax.scatter(X[y==1,0], X[y==1,1], marker='s', facecolors='black',
edgecolors='black', label='Class 1')

    ax.set_title(title)
    ax.legend()

##
# Step 3: Train KNN with different K values and visualize
##
k_values = [1, 5, 15]
fig, axes = plt.subplots(1, 3, figsize=(18,5))

for i, k in enumerate(k_values):
    knn = KNeighborsClassifier(n_neighbors=k)
    knn.fit(X_train_scaled, y_train)
    y_pred = knn.predict(X_test_scaled)
    acc = accuracy_score(y_test, y_pred)

    plot_decision_boundary(knn, X_test_scaled, y_test, axes[i],
                           title=f'K={k}, Test Accuracy={acc:.2f}')

plt.tight_layout()
plt.show()

```

Output :

